Übungen zur Vorlesung
Softwaretechnik – Wintersemester 2025/2026

Übungsblatt 13

Zu bearbeiten bis Mittwoch, 28.01.

1 Entwurfsmuster

Beantworten Sie kurz und bündig folgende Fragen. Sie finden die entsprechenden Informationen im Lesestoff GoF Kapitel 1.5.

- Welche Vorteile ergeben sich durch den Einsatz eines Entwurfsmusters?

  Wiederverwendung von erprobten Lösungen, Qualität (Verständlichkeit, Wartbarkeit), Kostenersparnis, abstrakte Dokumentation von Entwürfen, gemeinsames Vokabular zur Verständigung

- Entwurfsmuster werden in drei Kategorien unterteilt. Nennen Sie die Kategorien sowie jeweils ein passendes Entwurfsmuster.

  Creational: Factory Method, Builder
  Structural: Facade
  Behavioural: Iterator, State, Strategy, Visitor

- Manche Entwurfsmuster werden häufig miteinander kombiniert. Geben Sie hierfür zwei Beispiele an und erklären diese kurz.

  Singleton + Factory Method, Composite + Visitor, Composite + Iterator

2 Entwurf von Klassenhierarchien: Composite Pattern

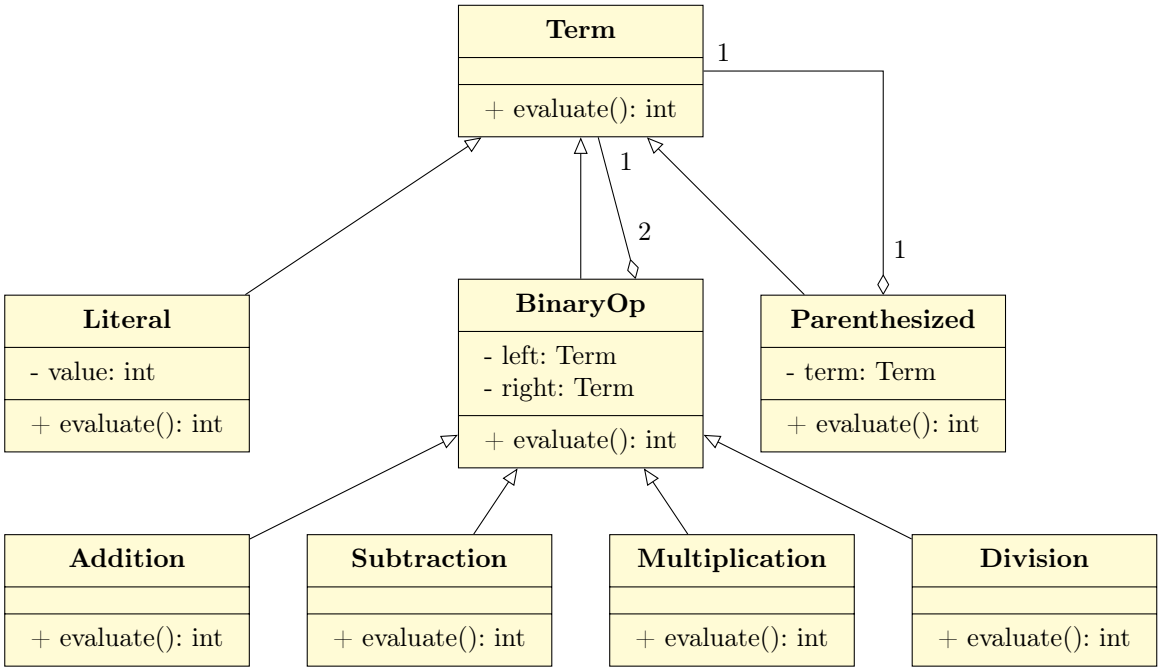
Dieser Artikel bietet ihnen Informationen zum Composite Pattern: `https://refactoring.guru/design-patterns/composite`.

Wir betrachten in dieser Aufgabe Terme über die Rechenarten $\circ \in \{+, -, \cdot, /\}$, die rekursiv definiert sind:

- jedes Literal ist ein Term, z.B. „4“.
- ist t ein Term, so ist „(t)“ ein (geklammerter) Term.
- sind t_1, t_2 Terme, so ist „$t_1 \circ t_2$“ ebenso ein Term.

Beispiele für gültige Terme sind also „$4 + 8$“, „$4 \cdot 8$“ oder „$4 + (4 \cdot 8)$“.

a) Modellieren Sie die Klassenstruktur in UML auf Basis des Composite Pattern. Terme werden rekursiv zusammengesetzt. Sehen Sie mindestens einzelne Klassen für die *Addition* und *Multiplikation* vor, sowie weitere Klassen für *geklammerte Terme* und *Literale*, welche ganze Zahlen repräsentieren. Ein Term kann über eine Methode `evaluate()` ausgewertet werden.



3 Decorator

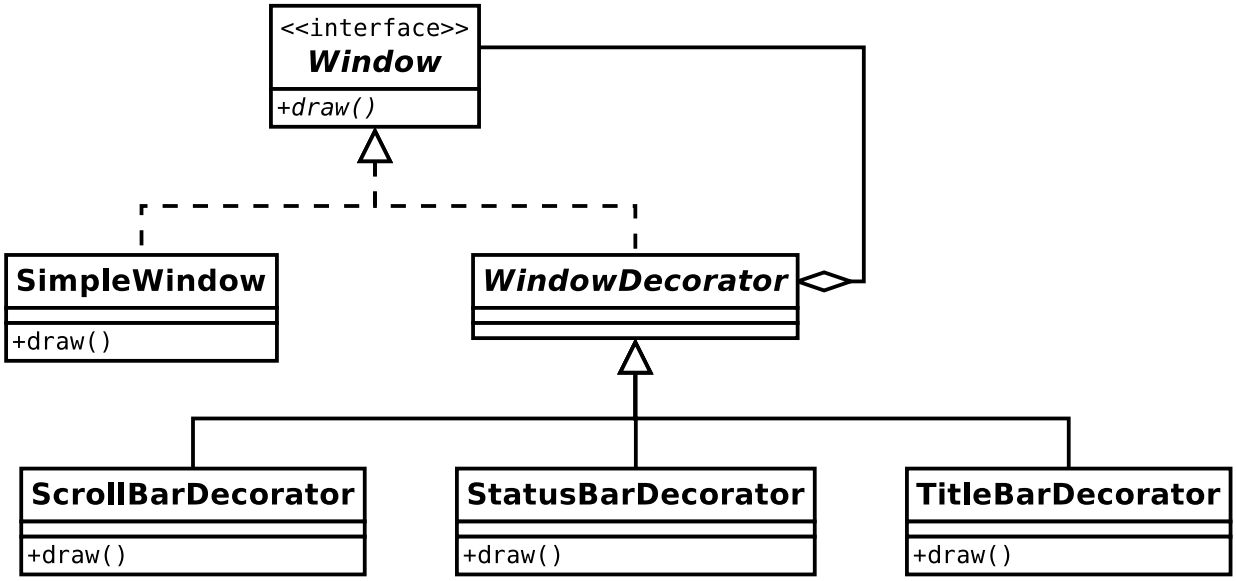
Dieser Artikel bietet ihnen Informationen zum Decorator Pattern: `https://refactoring.guru/design-patterns/decorator`.

Modellieren Sie folgenden Sachverhalt als UML-Klassendiagramm auf Basis des Decorator Pattern.

Eine Grafikbibliothek soll die Verwendung folgender Fenstertypen erlauben:

- einfache Fenster ohne Zusatzfunktionalität
- Fenster, die eine Titelleiste haben
- Fenster, die eine Statusleiste haben
- Fenster, die horizontal und vertikal „scrollbar“ sind
- alle daraus konstruierbaren „Featurekombinationen“, wie z.B. ein Fenster mit Titelleiste das horizontal und vertikal „scrollbar“ ist

Eine Implementierung alle Features in der Window-Klasse würde die Komplexität dieser Klasse zu stark ansteigen lassen. Eine Unterklasse von Window für jede Feature-Kombination führt zu einer exponentiellen Explosion von Klassen. Eine Lösung über eine Klasse „WindowFeature“ oder „Component“, von denen ein Window mehrere besitzen kann, kommt nicht ohne Speziallogik für jedes Feature in der Windows-Klasse aus, womit wieder das ursprüngliche Problem entsteht.

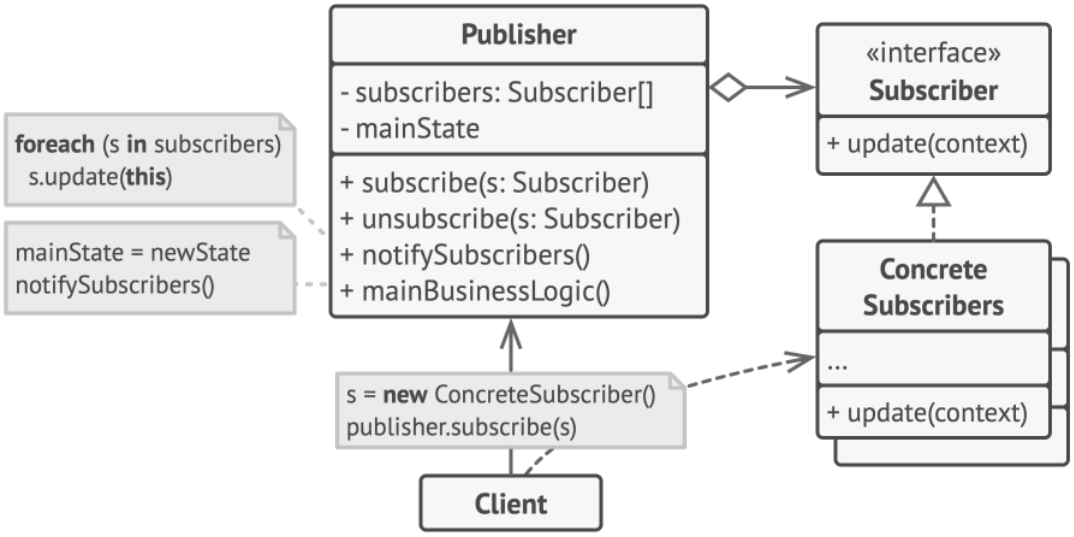


Eine Beispielimplementierung finden Sie im Anhang.

4 Observer Pattern

Dieser Artikel bietet ihnen Informationen zum Observer Pattern: `https://refactoring.guru/design-patterns/observer`.

Stellen Sie das Beobachter Entwurfsmuster (Observer-Pattern) in einem UML Klassendiagramm dar (inkl. wichtige Methoden, aber ohne deren Inhalt). Multiplizitäten können weggelassen werden. Verwenden Sie Interfaces wenn diese sinnvoll sind.



5 Memento Pattern

Dieser Artikel bietet ihnen Informationen zum Memento Pattern: `https://refactoring.guru/design-patterns/memento`. In dieser Aufgabe arbeiten Sie mit dem Memento Pattern, welches für die Verwirklichung von Undo-Funktionalität nützlich ist.

Szenario

Änderungen am Speiseplan in der Mensa-App sollen rückgängig gemacht werden können. Dazu soll es möglich sein, zu jedem vorher gespeicherten Zustand des Speiseplans zurückzuspringen. Zustände werden über das Datum und den Zeitpunkt des Speicherns (Typ DateTime) gespeichert.

Zunächst zeigt die folgende Skizze einen **schlechten Entwurf**: Alle privaten Felder des Speiseplan werden public gemacht. Bei einem Restore eines alten Memento setzt der externe SpeiseplanManager einfach alle Werte im Speiseplan von außen auf die ursprünglichen Werte.

```
class Speiseplan {
  public Gericht montag;
  public Gericht dienstag;
  public Gericht mittwoch;
  public Gericht donnerstag;
  public Gericht freitag;

  // ...
}

record SpeiseplanSnapshot(
  Gericht montag,
  Gericht dienstag,
  Gericht mittwoch,
  Gericht donnerstag,
  Gericht freitag
) {};

class SpeiseplanManager {
  private Map<DateTime, SpeiseplanSnapshot> snapshots;

  public void save(Speiseplan plan) {
    SpeiseplanSnapshot snapshot = new SpeiseplanSnapshot(
      plan.montag,
      plan.dienstag,
      plan.mittwoch,
      plan.donnerstag,
      plan.freitag
    );
    snapshots.put(currentDateTime(), snapshot);
  }

  public void restore(Speiseplan plan, DateTime savedAt) {
    // Assume savedAt always exists in 'snapshots'
    SpeiseplanSnapshot snapshot = snapshots.get(savedAt);
    plan.montag = snapshot.montag();
    plan.dienstag = snapshot.dienstag();
    plan.mittwoch = snapshot.mittwoch();
    plan.donnerstag = snapshot.donnerstag();
    plan.freitag = snapshot.freitag();
  }
}
```

a) Erklären Sie, was an der vorgeschlagenen Lösung im Bezug auf Wartbarkeit und zukünftige Änderungen am Speiseplan schlecht ist.

Der schlechte Entwurf verletzt das Encapsulation-Prinzip:

- Alle privaten Felder des Speiseplan sind public, was die interne Struktur offenlegt.
- Der externe SpeiseplanManager muss die genaue interne Struktur des Speiseplan kennen.
- Wenn der Speiseplan in Zukunft um neue Felder erweitert wird oder sich die interne Struktur ändert, muss auch der SpeiseplanManager angepasst werden.
- Die Speichern- und Wiederherstellungslogik ist über mehrere Klassen verteilt, was wartungsaufwändig ist.
- Es gibt keine Kontrolle darüber, dass die Zustände konsistent bleiben.

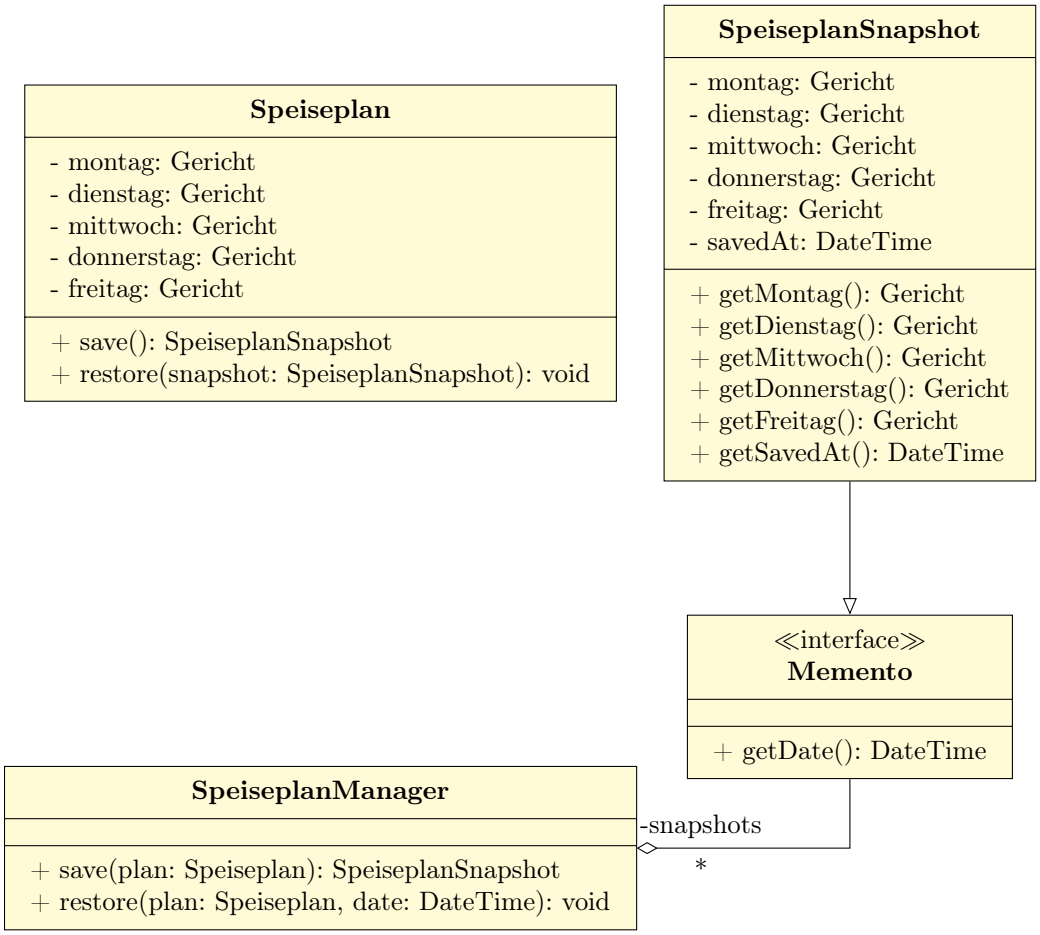
b) Entwerfen Sie ein Klassendiagramm für einen besseren Entwurf, der das Memento-Pattern nutzt.

Das Memento-Pattern löst das Problem, indem:

- Der Speiseplan selbst verantwortlich für die Erstellung von Snapshots (Memento) seiner internen Zustände ist.
- Der Speiseplan Zugriff auf alle seine privaten Felder hat und diese in ein Memento-Objekt encapsuliert.
- Der SpeiseplanManager (Caretaker) nur über das öffentliche Interface des Memento kommuniziert und die internen Details nicht kennt.
- Das Memento-Objekt eine Kopie des Zustands speichert und diese bei Bedarf dem Speiseplan zurückgibt.
- Zukünftige Änderungen am Speiseplan keine Änderungen am SpeiseplanManager erfordern.

Die Struktur entspricht dem in der Referenz beschriebenen Pattern mit:

- **Originator (Speiseplan)**: Erstellt Memento-Objekte und kann seinen Zustand aus ihnen wiederherstellen.
- **Memento (SpeiseplanSnapshot)**: Speichert den Zustand des Speiseplan zum Zeitpunkt der Erstellung (inkl. Datum/Uhrzeit).
- **Caretaker (SpeiseplanManager)**: Verwaltet eine Sammlung von Memento-Objekten und orchestriert das Speichern und Wiederherstellen.



Allgemeine Hinweise:

- Die vorgestellte Lösung ist lediglich ein Lösungsvorschlag. Es wird kein Anspruch auf Vollständigkeit oder Korrektheit erhoben.
- Das Blatt bereitet Sie auf die Vorlesung am **Mittwoch, 28.01.** vor.
- Das Blatt wird in der Übung am **29.01.** besprochen.
- Wir empfehlen Ihnen auch das Quiz in Moodle zu bearbeiten.